

DDR

Introduction

Goal: Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota.

Requirements

Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota. Il DDR si deve fermare quando tornato in HOME.

Il DDR è anche sensibile ai dati rilevati da un Radar. Quando il Radar rileva un 'intruso' a una distanza minore di una distanza prefissata ``DMIN``:

1. se il DDR si sta muovendo, il DDR si ferma fino a che il Radar non rileva più un 'intruso' così vicino
2. se il DDR è fermo in HOME, il DDR ruota su sè stesso fino a che un 'intruso' così vicino scompare

Requirement analysis

Quali sono le entità del sistema?

1. **HOME** per il committente è la posizione iniziale dove si trova il Robot all'interno della stanza.

Partiamo dalla formalizzazione della stanza:

Possiamo modellare la stanza come un rettangolo in un piano con base B e altezza H, dove entrambe le misure sono espresse in metri.

Nel piano il Robot occuperà un'area, non un punto, che assumo quadrata di lato R.

Posso quindi misurare la stanza in unità robotiche.

Posso quindi considerare l'unità robotica come una cella della mia stanza che quindi posso rappresentare come una griglia.

Per rappresentare la stanza usiamo 3 variabili: B, H, R :

```
var B //base  
var H //altezza  
var R //unità robotica
```

Per rappresentare HOME usiamo una coppia di coordinate (xhome e yhome):

```
var xhome =0  
var yhome =0
```

2. **Radar** è un' entità proattiva che rileva la presenza di intrusi a una distanza minore di DMIN (distanza prefissata) misurata in metri.

Il radar può fare due cose: rispondere alle richieste oppure emettere eventi. Nel primo caso comunica la distanza rilevata in risposta ad una richiesta del sistema, mentre nel

secondo caso deve mandare un' informazione con la distanza rilevata e non potendo conoscere i destinatari emetterà un evento.

Formalizziamo il radar come un attore.

1° caso

```
System ddr

/*
 * DDR.qak
 */

//definizione dei messaggi
Event sonardata : distance(x) //distanza in metri

Context ctxddr ip [ host="localhost"    port=8040 ]

//definiamo attore radar
QActor radar context ctxddr {
  [#
    var D = 0
    var Timer = 1000L
  #]
  State s0 initial {
    println("$name | go") color magenta
  }
  Goto distanza

  State distanza{
    emit sonardata : distance(x)
  }
  Transition t0
    whenTimeVar Timer -> distanza
}
```

2° caso

```
System ddr

/*
 * DDR.qak
 */

// request-reply
Request getSonarDistance : getSonarDistance(x) "richiesta del sistema"
Reply  radar : distance(D) for getSonarDistance

//contesto del sistema
Context ctxDDR ip [ host="localhost"    port=8040 ]

//definiamo attore Radar
QActor radar context ctxDDR {
  [#
    var D = 0
    var Timer = 1000L
  #]
  State s0 initial {
    println("$name | go") color magenta
```

```

    }
    Goto distanza

    State distanza{

    }
    Transition t0
        whenTimeVar Timer -> distanza
        whenRequest getSonarDistance -> handleRequest

    State handleRequest{
        onMsg( getSonarDistance : getSonarDistance(D) ){
            replyTo getSonarDistance with radar : distance( $D )
        }
    }
    Transition t0
        whenTimeVar Timer -> distanza
    }
}

```

3. **Differential Drive Robot (DDR)** è un'entità reattiva, priva della capacità di fare cose: è un semplice attuatore di comandi. Il DDR si può muovere in avanti e ruotare su sè stesso di 90°, inoltre conosce il perimetro della stanza. Lo modelliamo come servizio. Utilizzando il linguaggio QAK, che ci permette di ridurre l'abstraction gap, modello il robot come servizio.
- Il robot è fornito dal committente stesso, perciò rispettiamo le specifiche date.

```

System ddr
/*
 * DDR.qak
 */

//contesto del sistema
Context ctxDDR ip [ host="localhost" port=8040 ]
Context ctxDDRgrid ip [ host="127.0.0.1" port=8050 ]

Event radar : distance(x) //distanza in metri

//definiamo attore Radar
QActor radar context ctxDDR {

}

```

Problem analysis

Test plans

Project

Testing

Deployment

Maintenance

By Marilena Rugiero email: marilena.rugiero@studio.unibo.it,
uni/softwareEngineering



GIT repo: <https://github.com/marilena-uni/softwareEngineering>